



PROYECTO INTERMODULAR 2º DAM

Juego misiles

Contenido

1.	INFORMACIÓN GENERAL DEL PROYECTO	3
2.	MODELO DE SEGURIDAD	4
2.1.	Arquitectura de Autenticación.....	4
2.2.	Principios de Seguridad	4
3.	ACTORES DEL SISTEMA	5
4.	MÓDULOS DEL SISTEMA	6
5.	REQUISITOS FUNCIONALES DETALLADOS.....	7
5.1.	RF-01: Autenticación del Middleware	7
5.4.	RF-04: Listar Usuarios.....	9
5.5.	RF-05: Consultar Usuario por ID.....	9
5.6.	RF-06: Actualizar Usuario.....	9
5.7.	RF-07: Eliminar Usuario	10
5.8.	RF-08: Consultar Planetas del Usuario	10
5.9.	RF-09: Comprar Planeta para Usuario.....	10
5.10.	RF-10: Crear Planeta (Con usuariold en Body)	11
5.11.	RF-11: Listar Todos los Planetas	11
5.12.	RF-12: Ranking de Planetas	12
5.13.	RF-13: Actualizar Planeta	12
5.14.	RF-14: Eliminar Planeta	12
5.15.	RF-15: Guardar Partida.....	13
5.16.	RF-16: Finalizar Partida	14
5.17.	RF-17: Finalizar Partida con Posiciones.....	14
5.18.	RF-18: Listar Partidas	15
5.19.	RF-19: Consultar Partida por ID	15
5.20.	RF-20: Copiar Partidas a MongoDB.....	15
5.21.	RF-21: Listar Partidas de MongoDB.....	15
5.22.	RF-22: Usuario con Más Victorias.....	16
5.23.	RF-23: Tipo de Planeta con Más Victorias	16
5.24.	RF-24: Ranking de Usuarios	16
5.25.	RF-25: Ranking de Tipos de Planeta	16
6.	REQUISITOS NO FUNCIONALES	17
7.	MODELO DE DATOS	18
8.	DIAGRAMA DE CASOS DE USO	19
9.	RESUMEN DE ENDPOINTS API	20
10.	RUBRICA.....	23

1. INFORMACIÓN GENERAL DEL PROYECTO

Nombre: Planetas AD v2

Tipo: API REST para Middleware

Tecnologías principales:

- Spring Boot 4.0.2
- Java 20+
- MySQL (base principal)
- MongoDB Atlas (estadísticas)

Arquitectura: MVC con capa de servicios

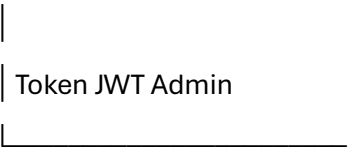
Seguridad: JWT – usuario administrador único

2. MODELO DE SEGURIDAD

2.1.Arquitectura de Autenticación

text

Usuario Final (Jugador) → Middleware (Cliente) → API REST (este sistema)



2.2.Principios de Seguridad

Principio	Descripción
Usuario único	Solo middleware_admin puede autenticarse
Token centralizado	Un único token para todas las operaciones del middleware
Endpoints protegidos	Todos requieren token (excepto /api/auth/login)
Identificación explícita	El usuariold siempre se envía como parámetro (no se infiere del token)

3. ACTORES DEL SISTEMA

Actor	Descripción
Middleware	Sistema externo autorizado – único cliente de la API
Administrador	middleware_admin – genera el token JWT inicial
Usuario (Jugador)	Entidad gestionada – nunca accede directamente a la API

4. MÓDULOS DEL SISTEMA

- 4.1 Módulo de Autenticación (Admin)
- 4.2 Módulo de Gestión de Usuarios
- 4.3 Módulo de Gestión de Planetas
- 4.4 Módulo de Gestión de Partidas
- 4.5 Módulo de Estadísticas (MongoDB)

5. REQUISITOS FUNCIONALES DETALLADOS

5.1.RF-01: Autenticación del Middleware

ID: RF-01 **Nombre:** Login de administrador (Middleware) **Módulo:** Autenticación

Prioridad: Crítica **Actor:** Middleware

Descripción: El sistema permite la autenticación exclusiva del usuario middleware_admin para obtener token JWT.

Datos de entrada:

Campo	Tipo	Obligatorio	Validación
nickname	String	Sí	Debe ser exactamente "middleware_admin"
password	String	Sí	Contraseña válida (BCrypt)

Datos de salida:

Campo	Tipo
token	String (JWT)
message	String

Endpoint: POST /api/auth/login

Reglas de negocio:

- RN-01.1 Solo middleware_admin puede autenticarse
- RN-01.2 Cualquier otro nickname → 401 Unauthorized
- RN-01.3 Expiración: 1 hora (3600 s)
- RN-01.4 Token obligatorio en peticiones posteriores (Bearer)

Ejemplo petición:

JSON

```
{
  "nickname": "middleware_admin",
  "password": "password_secreto"
}
```

Ejemplo respuesta:

JSON

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "message": "Middleware authenticated successfully"
}
```


5.2. RF-02: Cierre de Sesión

ID: RF-02 **Nombre:** Logout del middleware **Módulo:** Autenticación **Prioridad:** Baja **Actor:** Middleware

Descripción: Permite al middleware invalidar o cerrar la sesión actual (aunque en implementación JWT stateless suele ser solo un borrado local del token por parte del cliente).

Datos de entrada: — (ninguno en el body)

Datos de salida: (mensaje de confirmación o vacío con código 200/204)

Endpoint: POST /api/auth/logout

Requiere: Token JWT en header Authorization: Bearer {token}

Reglas de negocio:

- RN-02.1: El endpoint solo valida que exista un token válido
- RN-02.2: No existe lista de tokens revocados (stateless) → el logout es principalmente del lado cliente

5.3. RF-03: Crear Usuario (Jugador)

ID: RF-03 **Nombre:** Registro de jugador **Módulo:** Gestión de Usuarios **Prioridad:** Alta **Actor:** Middleware

Descripción: El middleware crea un nuevo jugador en el sistema en nombre del juego.

Datos de entrada:

Campo	Tipo	Obligatorio	Validación
-------	------	-------------	------------

nombre	String	Sí	No vacío
--------	--------	----	----------

apellidos	String	Sí	No vacío
-----------	--------	----	----------

nickname	String	Sí	Único, no vacío
----------	--------	----	-----------------

password	String	Sí	No vacío
----------	--------	----	----------

email	String	Sí	Formato email válido, único
-------	--------	----	-----------------------------

Datos de salida:

Campo	Tipo
-------	------

id	Long
----	------

nickname	String
----------	--------

Campo Tipo

monedas Integer

planetas Array

Endpoint: POST /api/usuarios

Requiere: Token JWT

Reglas de negocio:

- RN-03.1: La contraseña se almacena encriptada con BCrypt
- RN-03.2: El nickname debe ser único en el sistema
- RN-03.3: El email debe ser único en el sistema
- RN-03.4: El usuario inicia con 0 monedas
- RN-03.5: **NO** se genera ni se devuelve token para el jugador creado

5.4.RF-04: Listar Usuarios

ID: RF-04 **Nombre:** Listado de usuarios **Módulo:** Gestión de Usuarios **Prioridad:** Media
Actor: Middleware

Descripción: Devuelve la lista completa de usuarios registrados (jugadores).

Datos de entrada: — (paginación opcional en futuras versiones)

Datos de salida: Lista de objetos usuario (id, nickname, email, monedas, etc.)

Endpoint: GET /api/usuarios

Requiere: Token JWT

5.5.RF-05: Consultar Usuario por ID

ID: RF-05 **Nombre:** Obtener información de usuario **Módulo:** Gestión de Usuarios
Prioridad: Media **Actor:** Middleware

Descripción: Obtiene los datos detallados de un usuario específico.

Endpoint: GET /api/usuarios/{id}

Requiere: Token JWT

5.6.RF-06: Actualizar Usuario

ID: RF-06 **Nombre:** Modificar datos de usuario **Módulo:** Gestión de Usuarios **Prioridad:** Media
Actor: Middleware

Descripción: Permite modificar alguno o varios campos de un usuario existente.

Endpoint: PUT /api/usuarios/{id}

Requiere: Token JWT

5.7.RF-07: Eliminar Usuario

ID: RF-07 **Nombre:** Eliminar usuario **Módulo:** Gestión de Usuarios **Prioridad:** Baja **Actor:** Middleware

Descripción: Elimina permanentemente un usuario y sus relaciones (planetas, etc.) según reglas de borrado en cascada.

Endpoint: DELETE /api/usuarios/{id}

Requiere: Token JWT

5.8.RF-08: Consultar Planetas del Usuario

ID: RF-08 **Nombre:** Listar planetas de un jugador **Módulo:** Gestión de Planetas **Prioridad:** Alta **Actor:** Middleware

Descripción: Devuelve todos los planetas que posee un usuario específico.

Endpoint: GET /api/usuarios/{id}/planetas

Requiere: Token JWT

Reglas de negocio:

- RN-08.1: El middleware puede consultar los planetas de **cualquier** usuario
 - RN-08.2: No existe restricción de "solo mis planetas"
-

5.9.RF-09: Comprar Planeta para Usuario

ID: RF-09 **Nombre:** Compra de planeta **Módulo:** Gestión de Planetas **Prioridad:** Alta **Actor:** Middleware

Descripción: Asigna un nuevo planeta a un usuario descontando el coste en monedas.

Datos de entrada (body):

Campo	Tipo	Obligatorio
nombre	String	Sí
tipo	String	Sí

Endpoint: POST /api/usuarios/{id}/comprar-planeta

Requiere: Token JWT

Reglas de negocio:

- RN-09.1: El usuario se identifica exclusivamente por {id} en la URL
- RN-09.2: Verificar monedas suficientes antes de la compra
- RN-09.3: Costes según tipo:

Tipo	Coste (monedas)
Normal	200
Fuego	300
Agua	300
Planta	300
Aire	350
Roca	500

- RN-09.4: El planeta inicia con **5 vidas**
- RN-09.5: Se descuentan las monedas automáticamente

5.10. RF-10: Crear Planeta (Con usuariold en Body)

ID: RF-10 **Nombre:** Crear planeta directo **Módulo:** Gestión de Planetas **Prioridad:** Media **Actor:** Middleware

Descripción: Forma alternativa de crear un planeta indicando el usuario dueño en el body.

Datos de entrada:

Campo	Tipo	Obligatorio
usuariold	Long	Sí
nombre	String	Sí
tipo	String	Sí

Endpoint: POST /api/planetas

Requiere: Token JWT

5.11. RF-11: Listar Todos los Planetas

ID: RF-11 **Nombre:** Listado global de planetas **Módulo:** Gestión de Planetas **Prioridad:** Media **Actor:** Middleware

Descripción: Devuelve la lista completa de todos los planetas registrados en el sistema (independientemente del propietario).

Datos de entrada: — (posiblemente parámetros de paginación/filtro en futuras iteraciones)

Datos de salida: Lista de objetos planeta (id, nombre, tipo, vidas, victorias, propietario, etc.)

Endpoint: GET /api/planetas

Requiere: Token JWT

5.12. RF-12: Ranking de Planetas

ID: RF-12 **Nombre:** Ranking por victorias **Módulo:** Gestión de Planetas **Prioridad:** Alta **Actor:** Middleware

Descripción: Devuelve un ranking de planetas ordenados por número de victorias (descendente).

Datos de entrada:

Datos de salida: Lista ordenada de planetas con sus estadísticas (nombre, tipo, victorias, propietario, etc.)

Endpoint: GET /api/planetas/ranking

Requiere: Token JWT

5.13. RF-13: Actualizar Planeta

ID: RF-13 **Nombre:** Modificar planeta **Módulo:** Gestión de Planetas **Prioridad:** Baja **Actor:** Middleware

Descripción: Permite modificar atributos de un planeta existente (nombre, tipo, vidas, etc.).

Datos de entrada: Campos a actualizar (parcial o completo)

Endpoint: PUT /api/planetas/{id}

Requiere: Token JWT

5.14. RF-14: Eliminar Planeta

ID: RF-14 **Nombre:** Eliminar planeta **Módulo:** Gestión de Planetas **Prioridad:** Baja **Actor:** Middleware

Descripción: Elimina un planeta específico de la base de datos (y sus referencias si aplica).

Endpoint: DELETE /api/planetas/{id}

Requiere: Token JWT

5.15. RF-15: Guardar Partida

ID: RF-15 **Nombre:** Guardar estado de partida **Módulo:** Gestión de Partidas **Prioridad:** Alta **Actor:** Middleware

Descripción: Almacena el estado completo de una partida en curso para poder retomarla posteriormente.

Datos de entrada (ejemplo):

JSON

```
{
  "partida": {
    "estado": "EN_CURSO",
    "numeroRonda": 1
    // otros campos relevantes del estado de la partida
  },
  "participantes": [
    {
      "usuario": {"id": 1},
      "planeta": {"id": 1},
      "vida": 5
    },
    {
      "usuario": {"id": 2},
      "planeta": {"id": 2},
      "vida": 5
    }
  ]
}
```

Endpoint: POST /api/partidas/guardar

Requiere: Token JWT

5.16. RF-16: Finalizar Partida

ID: RF-16 **Nombre:** Finalizar partida simple **Módulo:** Gestión de Partidas **Prioridad:** Alta
Actor: Middleware

Descripción: Finaliza una partida indicando un ganador y un perdedor (modo 1 vs 1).

Endpoint: POST /api/partidas/finalizar

Requiere: Token JWT

Reglas de negocio:

- RN-16.1: El estado de la partida cambia a "FINALIZADO"
 - RN-16.2: El ganador recibe +200 monedas
 - RN-16.3: El perdedor recibe +50 monedas
 - RN-16.4: Los planetas perdedores pierden 1 vida
 - RN-16.5: El planeta ganador incrementa su contador de victorias
-

5.17. RF-17: Finalizar Partida con Posiciones

ID: RF-17 **Nombre:** Finalizar partida con ranking **Módulo:** Gestión de Partidas **Prioridad:** Alta
Actor: Middleware

Descripción: Finaliza una partida multi-jugador indicando la posición final de cada participante.

Datos de entrada (ejemplo):

JSON

```
{
  "idPartida": 1,
  "jugadores": [
    {"idJugador": 1, "posicion": 1},
    {"idJugador": 2, "posicion": 2},
    {"idJugador": 3, "posicion": 3}
  ]
}
```

Endpoint: POST /api/partidas/finalizar-con-posiciones

Requiere: Token JWT

Reglas de negocio:

- RN-17.1: Sistema de premios: $500 - 50 \times (\text{posición} - 1)$ monedas

- RN-17.2: Solo el 1º puesto no pierde vida
- RN-17.3: El planeta del 1º puesto incrementa su contador de victorias

5.18. RF-18: Listar Partidas

ID: RF-18 **Nombre:** Listado de partidas **Módulo:** Gestión de Partidas **Prioridad:** Media **Actor:** Middleware

Descripción: Devuelve la lista de partidas registradas (en curso, finalizadas, etc.).

Endpoint: GET /api/partidas

Requiere: Token JWT

5.19. RF-19: Consultar Partida por ID

ID: RF-19 **Nombre:** Detalle de partida **Módulo:** Gestión de Partidas **Prioridad:** Media **Actor:** Middleware

Descripción: Obtiene información detallada de una partida específica.

Endpoint: GET /api/partidas/{id}

Requiere: Token JWT

5.20. RF-20: Copiar Partidas a MongoDB

ID: RF-20 **Nombre:** Sincronización MySQL → MongoDB **Módulo:** Estadísticas **Prioridad:** Alta **Actor:** Middleware

Descripción: Copia o sincroniza datos de partidas desde MySQL hacia MongoDB Atlas para análisis estadístico.

Endpoint: POST /api/estadisticas/copiar

Requiere: Token JWT

5.21. RF-21: Listar Partidas de MongoDB

ID: RF-21 **Nombre:** Listado estadístico **Módulo:** Estadísticas **Prioridad:** Media **Actor:** Middleware

Descripción: Consulta partidas almacenadas en MongoDB (optimizado para lecturas analíticas).

Endpoint: GET /api/estadisticas/partidas

Requiere: Token JWT

5.22. RF-22: Usuario con Más Victorias

ID: RF-22 **Nombre:** Top usuario ganador **Módulo:** Estadísticas **Prioridad:** Alta **Actor:** Middleware

Descripción: Devuelve el usuario con mayor número de victorias (planetas ganadoras).

Endpoint: GET /api/estadisticas/usuario-top

Requiere: Token JWT

5.23. RF-23: Tipo de Planeta con Más Victorias

ID: RF-23 **Nombre:** Top tipo de planeta **Módulo:** Estadísticas **Prioridad:** Alta **Actor:** Middleware

Descripción: Devuelve el tipo de planeta con mayor número de victorias acumuladas.

Endpoint: GET /api/estadisticas/tipo-planeta-top

Requiere: Token JWT

5.24. RF-24: Ranking de Usuarios

ID: RF-24 **Nombre:** Ranking completo de usuarios **Módulo:** Estadísticas **Prioridad:** Media **Actor:** Middleware

Descripción: Lista ordenada de usuarios según criterio estadístico (victorias, monedas, etc.).

Endpoint: GET /api/estadisticas/ranking-usuarios

Requiere: Token JWT

5.25. RF-25: Ranking de Tipos de Planeta

ID: RF-25 **Nombre:** Ranking de tipos de planeta **Módulo:** Estadísticas **Prioridad:** Media **Actor:** Middleware

Descripción: Lista ordenada de tipos de planeta según victorias o rendimiento general.

Endpoint: GET /api/estadisticas/ranking-tipos-planeta

Requiere: Token JWT

6. REQUISITOS NO FUNCIONALES

RNF-01: Seguridad

- JWT con usuario admin único
- Contraseñas BCrypt
- Único endpoint público: /api/auth/login

RNF-02: Persistencia

- MySQL 8.0 (datos transaccionales)
- MongoDB Atlas (estadísticas / lecturas analíticas)
- ORM: Hibernate/JPA

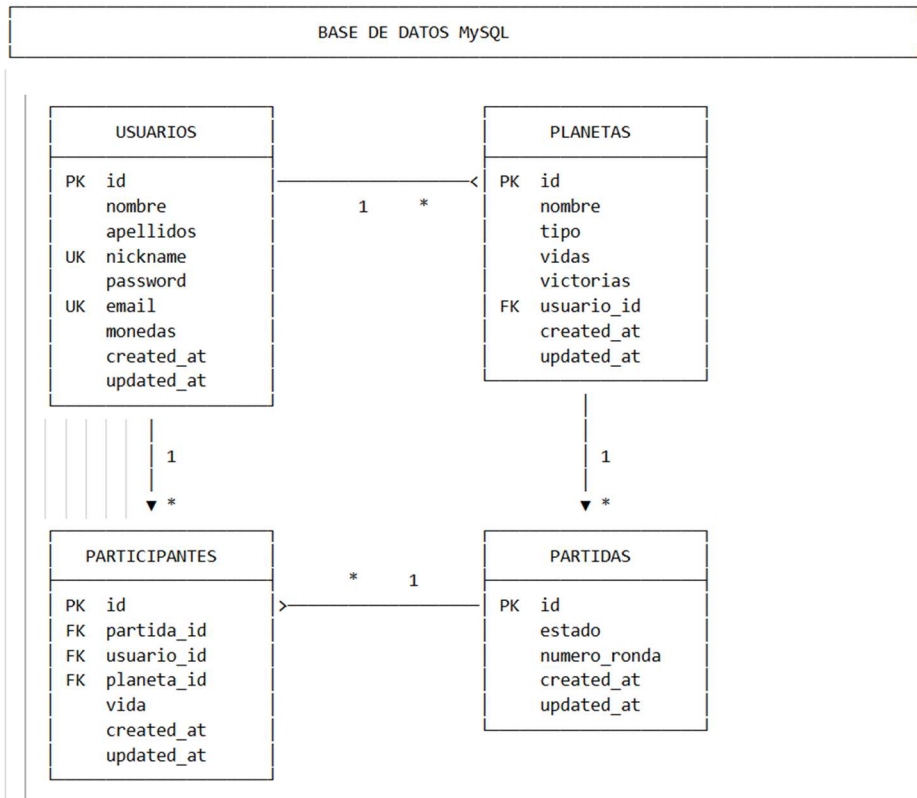
RNF-03: Rendimiento

- Pool de conexiones HikariCP
- JWT duración 1 hora

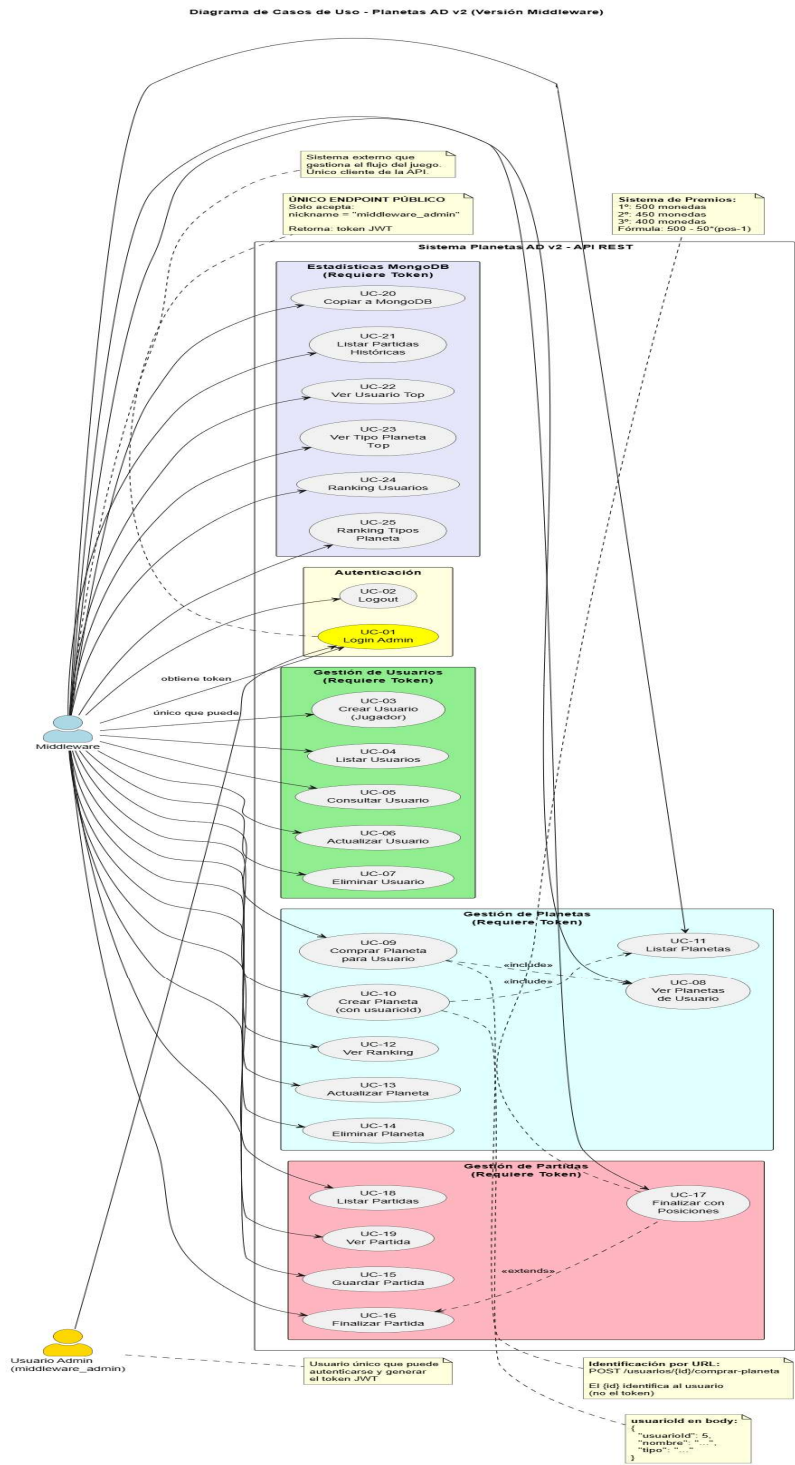
RNF-04: Disponibilidad

- Puerto: 8080 (por defecto)
- Empaquetado: WAR

7. MODELO DE DATOS



8. DIAGRAMA DE CASOS DE USO



9. RESUMEN DE ENDPOINTS API

Todos los endpoints (excepto el de login) requieren autenticación mediante **JWT** en el header:

Authorization: Bearer {token}

9.1. Autenticación

Método	Endpoint	Público	Descripción	Notas importantes
POST	/api/auth/login	Sí	Autenticación exclusiva del middleware	Solo acepta middleware_admin
POST	/api/auth/logout	No	Cierre de sesión (logout simbólico)	No revoca token (JWT stateless)

9.2. Usuarios

Método	Endpoint	Descripción	Requiere Token	Prioridad
POST	/api/usuarios	Crear nuevo jugador	Sí	Alta
GET	/api/usuarios	Listar todos los usuarios	Sí	Media
GET	/api/usuarios/{id}	Obtener datos de un usuario específico	Sí	Media
PUT	/api/usuarios/{id}	Actualizar datos de un usuario	Sí	Media
DELETE	/api/usuarios/{id}	Eliminar un usuario	Sí	Baja
GET	/api/usuarios/{id}/planetas	Listar planetas de un usuario específico	Sí	Alta
POST	/api/usuarios/{id}/comprar-planeta	Comprar / asignar nuevo planeta a usuario	Sí	Alta

9.3. Planetas

Método	Endpoint	Descripción	Requiere Token	Prioridad
--------	----------	-------------	----------------	-----------

POST	/api/planetas	Crear planeta (indicando usuariold en body)	Sí	Media
GET	/api/planetas	Listar todos los planetas del sistema	Sí	Media
GET	/api/planetas/ranking	Ranking de planetas por victorias	Sí	Alta
PUT	/api/planetas/{id}	Actualizar datos de un planeta	Sí	Baja
DELETE	/api/planetas/{id}	Eliminar un planeta	Sí	Baja

9.4. Partidas

Método	Endpoint	Descripción	Requiere Token	Prioridad
POST	/api/partidas/guardar	Guardar estado de una partida en curso	Sí	Alta
POST	/api/partidas/finalizar	Finalizar partida simple (1 vs 1)	Sí	Alta
POST	/api/partidas/finalizar-con-posiciones	Finalizar partida con posiciones/ranking	Sí	Alta
GET	/api/partidas	Listar todas las partidas	Sí	Media
GET	/api/partidas/{id}	Obtener detalle de una partida específica	Sí	Media

9.5. Estadísticas (MongoDB)

Método	Endpoint	Descripción	Requiere Token	Prioridad
POST	/api/estadisticas/copiar	Copiar / sincronizar partidas de MySQL → MongoDB	Sí	Alta
GET	/api/estadisticas/partidas	Listar partidas almacenadas en MongoDB	Sí	Media
GET	/api/estadisticas/usuario-top	Usuario con más victorias	Sí	Alta
GET	/api/estadisticas/tipo-planeta-top	Tipo de planeta con más victorias	Sí	Alta

GET	/api/estadisticas/ranking-usuarios	Ranking completo de usuarios	Sí	Media
GET	/api/estadisticas/ranking-tipos-planeta	Ranking de tipos de planeta	Sí	Media

10. RUBRICA

RF principal / Grupo	Criterio de evaluación específico	Reglas de Negocio (RN) clave a verificar	Puntos	Excelente (puntos completos)	Parcial (mitad o algo menos)	Insuficiente / No implementado
Autenticación (RF-01 y RF-02)	RF-01: Login exclusivo middleware_admin → token JWT generado correctamente	RN-01.1: Solo middleware_admin puede autenticarse RN-01.2: Otros nicknames → 401 RN-01.3: Expiración 1 hora RN-01.4: Token en peticiones posteriores	4	Todas RN cumplidas + BCrypt + claims correctos	Funciona pero falla alguna RN (ej. sin expiración)	No valida RN-01.1 o RN-01.2
	Todos los endpoints (excepto login) protegidos	RN general: Endpoints protegidos excepto login	4	401/403 correcto en todos	Algún endpoint accesible sin token	Sin protección real
	RF-02: Logout (simbólico)	No hay RN estrictas (stateless)	2	Endpoint existe y responde 200/204	Endpoint existe pero sin validación token	No implementado
Subtotal Autenticación			10			
Gestión de Usuarios (RF-03 a RF-07)	RF-03: POST /api/usuarios – Creación de jugador	RN-03.1: Password BCrypt RN-03.2: Nickname único RN-03.3: Email único RN-03.4: Inicia con 0 monedas RN-03.5: NO genera token	5	Todas RN cumplidas estrictamente	Crea usuario pero falla 1–2 RN (ej. monedas ≠0 o token devuelto)	Viola varias RN o no crea
	RF-04: GET /api/usuarios – Listado	No RN específicas	4	Lista completa y coherente	Lista parcial o con datos erróneos	No lista
	RF-05: GET /api/usuarios/{id}	No RN específicas	4	Detalle correcto	Detalle parcial	No encuentra
	RF-06: PUT /api/usuarios/{id}	No RN específicas (mantener unicidad si se cambia nick/email)	4	Actualiza sin romper constraints	Actualiza pero permite duplicados	No actualiza
	RF-07: DELETE /api/usuarios/{id}	No RN específicas	3	Elimina correctamente	Elimina pero deja inconsistencias	No elimina
Subtotal Usuarios			20			
Gestión de Planetas (RF-08 a RF-14)	RF-08: GET /api/usuarios/{id}/planetas	RN-08.1: Puede consultar cualquier usuario RN-08.2: Sin restricción propietario	5	Lista cualquier usuario sin restricción	Lista pero con restricción errónea	No lista

	RF-09: POST /api/usuarios/{id}/comprar-planeta	RN-09.1: Id en path (no token) RN-09.2: Verificar monedas suficientes RN-09.3: Costes exactos por tipo RN-09.4: Vidas iniciales = 5 RN-09.5: Descuenta monedas	5	Todas RN cumplidas + transacción atómica	Compra pero falla 1–2 RN (ej. sin validar monedas)	No valida costes o no descuenta
	RF-10: POST /api/planetas (usuariold en body)	No RN específicas adicionales	2	Crea correctamente	Crea con limitaciones	No implementado
	RF-11: GET /api/planetas	No RN específicas	3	Lista global correcta	Lista parcial	No lista
	RF-12: GET /api/planetas/ranking	No RN específicas (orden por victorias descendente implícito)	3	Ranking ordenado correctamente	Ordenado con errores	No ordenado
	RF-13 y RF-14: PUT/DELETE planeta	No RN específicas	2	Ambos funcionan	Solo uno o con errores	Ninguno
Subtotal Planetas			20			
Gestión de Partidas (RF-15 a RF-19)	RF-15: POST /api/partidas/guardar	No RN específicas detalladas	5	Guarda estado + participantes correctamente	Guarda incompleto	No guarda
	RF-16: POST /api/partidas/finalizar	RN-16.1: Estado → FINALIZADO RN-16.2: Ganador +200 monedas RN-16.3: Perdedor +50 monedas RN-16.4: Perdedor -1 vida RN-16.5: Ganador +1 victoria	6	Todas RN cumplidas + transaccional	Falla 1–2 RN (ej. monedas mal)	No actualiza correctamente
	RF-17: POST /api/partidas/finalizar-composiciones	RN-17.1: Premios $500 - 50 \times (\text{pos} - 1)$ RN-17.2: Solo 1º no pierde vida RN-17.3: 1º +1 victoria	6	Fórmula y reglas exactas	Premios parciales o vidas mal	No implementado o equivocado
	RF-18 y RF-19: GET partidas y detalle	No RN específicas	3	Listado y detalle correctos	Funcionan con fallos	No listan
Subtotal Partidas			20			
Estadísticas MongoDB (RF-20)	RF-20: POST /api/estadisticas/copiar	No RN específicas	5	Copia datos básicos al menos	Copia parcial	No implementado
RF-21	GET /api/estadisticas/partidas – listado MongoDB	2014	2	Lista correctamente desde Mongo	Lista parcial	No lista
RF-22	GET /api/estadisticas/usuario-top – usuario con más victorias	—	5	Top correcto y actualizado	Top parcial o erróneo	No implementado

RF-23	GET /api/estadisticas/tipo-planeta-top – tipo planeta top	—	4	Top correcto	Top parcial	No implementado
RF-24	GET /api/estadisticas/ranking-usuarios – ranking usuarios	—	2	Ranking ordenado correcto	Ranking parcial	No implementado
RF-25	GET /api/estadisticas/ranking-tipos-planeta – ranking tipos	—	2	Ranking ordenado correcto	Ranking parcial	No implementado
Subtotal Estadísticas			20			
Calidad y Buenas Prácticas	Arquitectura MVC limpia, excepciones globales, validaciones	No RN específicas	5	Capas separadas + @ControllerAdvice + @Valid	Arquitectura aceptable	Código mezclado
	Uso de @Transactional en operaciones críticas	No RN específicas	3	Transacciones en compra, finalización, etc.	Alguna transacción faltante	Sin transacciones
	Documentación (README, endpoints)	No RN específicas	2	README claro + ejemplos	README básico	Sin documentación
Subtotal Calidad			10			
TOTAL			100			